

针对经济时序数据的因果推断

李春阳

东北大学计算机科学与工程系

2026 年 1 月 3 日

摘要

本文聚焦于上一篇介绍的 STIC 和 ACD 的实现细节。在小样本、高噪声、非平稳等挑战下,传统方法难以准确捕捉动态因果关系。通过分析前人工作的核心思想,我使用 DGL 框架实现了 ACD 的简化版本,并基于短时不变性原理设计了局部因果扫描器。实验表明,在宏观经济数据集上,这些方法能够有效识别不同时期(如疫情、加息、复苏)的因果结构变化。本工作通过将理论与实践结合,验证了这些方法在真实场景中的实用价值,也为后续研究提供了一点小参考。https://github.com/Li973/ML_classwork

目录

1 背景与问题定义	2
1.1 时间序列因果发现的意义	2
1.2 问题的数学建模	2
2 ACD 方法的具体实现	3
2.1 ACD 框架概述	3
2.2 DGL 实现细节	3
2.2.1 基础组件实现	3
2.2.2 编码器实现	4
2.2.3 解码器实现	5
2.2.4 完整 ACD 模型	6
2.3 训练与推理过程	7
2.3.1 训练流程	7
2.3.2 推理与因果图预测	8

1	背景与问题定义	2
3	局部因果扫描器实现	9
3.1	设计原理	9
3.2	具体实现	9
3.3	事件分析模块	11
4	实验分析与结果	12
4.1	数据集与预处理	12
4.2	实验结果	12
4.3	局部因果扫描结果	13
5	结论	14

1 背景与问题定义

1.1 时间序列因果发现的意义

在金融、医疗和气候科学等领域，理解变量间的因果关系比仅仅发现相关性更为重要。例如，理解美联储加息如何影响汇率波动，或疾病症状如何随治疗方案变化，都需要准确的因果分析。然而，传统方法在处理时间序列数据时面临诸多挑战：

- **小样本问题**：月度宏观经济数据通常仅有 10-15 年（120-180 个时间点）
- **机制变化**：不同经济时期（如疫情、加息、复苏）因果结构可能不同
- **噪声干扰**：真实数据通常包含测量误差和外部干扰
- **隐藏混杂变量**：未观测变量可能导致虚假因果关系

1.2 问题的数学建模

给定 N 个变量的多元时间序列 $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T] \in \mathbb{R}^{N \times T}$, 其中 $\mathbf{x}_t = [x_{t,1}, x_{t,2}, \dots, x_{t,N}]^T$ 表示 t 时刻的观测值。时间序列因果发现任务是推断两个关键元素：

1. 因果图结构 $G = (V, E)$, 其中 V 是变量节点集合, E 是因果边集合
2. 时滞信息 $\tau_{i \rightarrow j}$, 表示变量 i 对 j 的影响延迟

在 Granger 因果框架下，如果使用变量 i 的历史值能显著提高对变量 j 未来值的预测精度，则认为 i 是 j 的 Granger 原因。

2 ACD 方法的具体实现

2.1 ACD 框架概述

Amortized Causal Discovery (ACD) 通过分离因果图预测与动力学建模，实现了跨样本知识共享。其核心框架包含两个组件：

- **编码器** $q_\phi(z|x)$ ：预测因果图结构
- **解码器** $p_\theta(x|z)$ ：在预测的因果图约束下模拟系统动力学

2.2 DGL 实现细节

以下使用 Deep Graph Library (DGL) 实现 ACD 的核心组件。安装命令大致如下：

```
1 pip install torch dgl dglgo pandas numpy scikit-learn
```

Listing 1: 部分依赖

当然 DGL 有挺麻烦的兼容性问题，首先 GPU 环境得针对 cuda 版本安装对应版本，其次它和 torch 等依赖也有很多版本冲突的坑，这边先用 CPU 版对付一下。

2.2.1 基础组件实现

首先实现 Gumbel-Softmax 采样，使离散边类型可微分：

```
1 import torch
2 import torch.nn.functional as F
3
4 def gumbel_softmax_sample(logits, tau=1.0, hard=False):
5     """
6     通过Gumbel-Softmax技巧实现离散变量的可微分采样
7
8     参数：
9         logits: [E, n_edge_types] 边类型的未归一化对数概率
10        tau: 温度参数，控制采样软硬度
11        hard: 是否返回离散的one-hot向量
12    """
13    gumbels = -torch.empty_like(logits).exponential_().log()
14    #加噪声并缩放
15    gumbels = (logits + gumbels) / tau
16    y = F.softmax(gumbels, dim=-1)
17
18    if hard:
19        index = y.max(-1, keepdim=True)[1]
20        y_hard = torch.zeros_like(logits).scatter_(-1, index, 1.0)
21        #保持梯度流动
```

```

22     y = y_hard - y.detach() + y
23     return y

```

Listing 2: Gumbel-Softmax 采样实现

2.2.2 编码器实现

ACD 编码器使用 GNN 处理全连接图，预测边类型：

```

1 import dgl
2 from dgl.nn import GraphConv
3
4 class CausalEncoder(nn.Module):
5     """基于GNN的因果图预测编码器"""
6
7     def __init__(self, input_dim, hidden_dim, n_edge_types):
8         super().__init__()
9         self.n_edge_types = n_edge_types
10
11         # 节点嵌入层
12         self.node_emb = nn.Linear(input_dim, hidden_dim)
13
14         # 两层GCN
15         self.gcn1 = GraphConv(hidden_dim, hidden_dim, activation=F.relu)
16         self.gcn2 = GraphConv(hidden_dim, hidden_dim, activation=F.relu)
17
18         # 边类型预测器
19         self.edge_logits = nn.Sequential(
20             nn.Linear(2 * hidden_dim, hidden_dim),
21             nn.ReLU(),
22             nn.Linear(hidden_dim, n_edge_types)
23         )
24
25     def forward(self, x, edge_index, tau=0.5, hard=False):
26         """
27         参数:
28             x: [N, T, D] 多变量时间序列
29             edge_index: [2, N*N] 全连接图的边索引
30             tau: Gumbel-Softmax 温度
31             hard: 是否使用hard采样
32         """
33         # 使用最后时间步作为节点特征
34         if x.dim() == 3:
35             x = x[:, -1, :] # [N, D]
36
37         # 节点嵌入

```

```

38     h = self.node_emb(x)  # [N, hidden]
39
40     # 构建DGL图
41     g = dgl.graph((edge_index[0], edge_index[1]), num_nodes=x.size(0))
42     g = dgl.add_self_loop(g)
43     g.ndata['h'] = h
44
45     # GNN消息传递
46     h = self.gcn1(g, h)
47     h = self.gcn2(g, h)
48
49     # 准备边特征
50     src, dst = edge_index
51     h_pairs = torch.cat([h[src], h[dst]], dim=-1)  # [N^2, 2*hidden]
52
53     # 预测边类型
54     logits = self.edge_logits(h_pairs)  # [N^2, n_edge_types]
55     z = gumbel_softmax_sample(logits, tau=tau, hard=hard)
56
57     return z, logits

```

Listing 3: ACD 编码器实现

2.2.3 解码器实现

```

1 class CausalDecoder(nn.Module):
2
3     def __init__(self, node_dim, hidden_dim, n_edge_types):
4         super().__init__()
5         self.n_edge_types = n_edge_types
6
7         # 每种边类型的消息函数
8         self.edge_fns = nn.ModuleList([
9             nn.Sequential(
10                 nn.Linear(2 * node_dim, hidden_dim),
11                 nn.ReLU(),
12                 nn.Linear(hidden_dim, node_dim)
13             ) for _ in range(n_edge_types)
14         ])
15         self.edge_fns[0] = lambda x: torch.zeros_like(x[:, :node_dim])
16
17         # 节点更新函数
18         self.node_update = nn.Sequential(
19             nn.Linear(2 * node_dim, hidden_dim),
20             nn.ReLU(),

```

```

21         nn.Linear(hidden_dim, node_dim)
22     )
23
24     def forward(self, x_t, z, edge_index):
25         """
26         参数:
27             x_t: [N, D] 当前时间步状态
28             z: [N^2, n_edge_types] 边类型概率
29             edge_index: [2, N^2] 边索引
30         """
31         src, dst = edge_index
32         agg_msg = torch.zeros_like(x_t) # [N, D]
33
34         #按边类型聚合消息
35         for e in range(self.n_edge_types):
36             weight_e = z[:, e].unsqueeze(-1) # [N^2, 1]
37             msg_input = torch.cat([x_t[src], x_t[dst]], dim=-1) # [N^2, 2D]
38             msg = self.edge_fns[e](msg_input) # [N^2, D]
39             weighted_msg = weight_e * msg
40
41         #按目标节点聚合
42         agg_msg.index_add_(0, dst, weighted_msg)
43
44         #节点更新
45         node_input = torch.cat([x_t, agg_msg], dim=-1) # [N, 2D]
46         delta = self.node_update(node_input) # [N, D]
47
48         #预测下一时间步 (残差连接)
49         return x_t + delta

```

Listing 4: ACD 解码器实现

2.2.4 完整 ACD 模型

将编码器和解码器组合为完整模型:

```

1 class AmortizedCausalDiscovery(nn.Module):
2
3     def __init__(self, input_dim, hidden_dim=64, n_edge_types=3, tau=0.5):
4         super().__init__()
5         self.n_edge_types = n_edge_types
6         self.tau = tau
7
8         #创建编码器和解码器
9         self.encoder = CausalEncoder(input_dim, hidden_dim, n_edge_types)
10        self.decoder = CausalDecoder(input_dim, hidden_dim, n_edge_types)

```

```

11
12     def build_fully_connected_edge_index(self, N):
13         i = torch.arange(N).repeat_interleave(N)
14         j = torch.arange(N).repeat(N)
15         return torch.stack([i, j]) # [2, N*N]
16
17     def forward(self, x_seq, hard=False):
18         """
19         前向传播
20
21         参数:
22             x_seq: [N, T, D] 输入时间序列
23             hard: 是否使用hard采样
24         """
25         N, T, D = x_seq.shape
26         edge_index = self.build_fully_connected_edge_index(N).to(x_seq.device)
27
28         # 编码因果图
29         z, logits = self.encoder(x_seq, edge_index, tau=self.tau, hard=hard)
30
31         # 解码下一时间步
32         recon_loss = 0
33         for t in range(T-1):
34             x_t = x_seq[:, t, :]
35             x_target = x_seq[:, t+1, :]
36             x_pred = self.decoder(x_t, z, edge_index)
37             recon_loss += F.mse_loss(x_pred, x_target)
38
39         recon_loss /= (T-1)
40
41         # KL散度正则化 (鼓励稀疏性)
42         prior = torch.tensor([0.7] + [0.3/(self.n_edge_types-1)]*(self.
n_edge_types-1))
43         prior = prior.to(z.device).unsqueeze(0).expand_as(z)
44         q = F.softmax(logits, dim=-1)
45         kl_loss = F.kl_div(q.log(), prior, reduction='batchmean')
46
47         return recon_loss, kl_loss, z

```

Listing 5: 完整 ACD 模型

2.3 训练与推理过程

2.3.1 训练流程

以下展示了 ACD 模型的训练过程：

```

1 #初始化模型与优化器
2 model = AmortizedCausalDiscovery(input_dim=1, hidden_dim=32, n_edge_types=3)
3 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
4
5 #训练循环
6 for epoch in range(100):
7     total_loss = 0
8
9     for batch in dataloader:
10         # batch: [B, N, T, D]
11         recon_loss, kl_loss, _ = model(batch)
12         loss = recon_loss + 0.1 * kl_loss # 平衡重构与稀疏性
13
14         optimizer.zero_grad()
15         loss.backward()
16         optimizer.step()
17
18         total_loss += loss.item()
19
20 print(f"Epoch {epoch}: Loss = {total_loss/len(dataloader):.4f}")

```

Listing 6: ACD 训练流程

2.3.2 推理与因果图预测

训练后，可使用模型预测因果图：

```

1 def predict_causal_graph(model, x_seq):
2     """
3     预测时间序列的因果图
4
5     参数：
6         model: 训练好的ACD模型
7         x_seq: [N, T, D] 输入序列
8
9     返回：
10         adj: [N, N] 预测的邻接矩阵
11     """
12     model.eval()
13     with torch.no_grad():
14         N = x_seq.shape[0]
15         edge_index = model.build_fully_connected_edge_index(N).to(x_seq.device)
16
17     #获取硬采样结果
18     z, _ = model.encoder(x_seq, edge_index, tau=0.1, hard=True)

```



```

19
20     #转换为邻接矩阵
21     edge_types = torch.argmax(z, dim=-1) # [N^2]
22     adj = edge_types.view(N, N) # [N, N]
23     #移除自环
24     adj.fill_diagonal_(0)
25
26     return adj.cpu().numpy()

```

Listing 7: 因果图预测

3 局部因果扫描器实现

3.1 设计原理

受 STIC 短时不变性原理启发，我们设计了局部因果扫描器 (LocalCausalScanner)，在滑动窗口内检测因果关系。这种方法特别适合小样本场景，如仅有 120 个月度观测值的宏观经济数据。

3.2 具体实现

以下是局部因果扫描器的完整实现：

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.linear_model import LinearRegression
4 from scipy.stats import pearsonr
5
6 class LocalCausalScanner:
7     """在滑动窗口内检测变量间的因果关系"""
8
9     def __init__(self, window=24, step=3, max_lag=4, pc_thresh=0.32):
10         self.window = window
11         self.step = step
12         self.max_lag = max_lag
13         self.pc_thresh = pc_thresh
14
15     def scan(self, df):
16         vars = df.columns.tolist()
17         n = len(df)
18         findings = []
19
20         #滑动窗口
21         for start in range(0, n - self.window + 1, self.step):
22             sub = df.iloc[start:start + self.window]

```

项
因

```

23     window_center = sub.index[len(sub) // 2]
24
25     #检查所有变量对
26     for src in vars:
27         for tgt in vars:
28             if src == tgt:
29                 continue
30
31     #检查不同时滞
32     for lag in range(1, self.max_lag + 1):
33         y = sub[tgt].iloc[lag:].values
34         x_auto = sub[tgt].shift(1).iloc[lag:].values #自回归
35
36         x_src = sub[src].shift(lag).iloc[lag:].values #潜在原
37
38         if len(y) < 10: #确保有足够的样本
39             continue
40
41         X = np.column_stack([x_auto, x_src])
42         reg = LinearRegression().fit(X, y)
43         coef = reg.coef_[1] #源变量的系数
44         if abs(coef) < 0.01:
45             continue
46
47         #移除目标变量的自相关影响
48         res_y = y - (reg.predict(X) - coef * x_src)
49         pc = pearsonr(x_src, res_y)[0]
50
51         if abs(pc) > self.pc_thresh:
52             findings.append({
53                 'src': src,
54                 'tgt': tgt,
55                 'lag': lag,
56                 'coef': round(coef, 3),
57                 'pcorr': round(pc, 3),
58                 'window_center': window_center,
59                 'window_start': sub.index[0],
60                 'window_end': sub.index[-1]
61             })
62
63     return pd.DataFrame(findings)

```

Listing 8: 局部因果扫描器实现

3.3 事件分析模块

为适应不同时期（如疫情、加息、复苏）的因果结构变化，设计了事件分析模块：

```

1 class EventAnalyzer:
2     #预定义的经济事件时间段
3     EVENTS = {
4         '疫情': ('2020-03', '2020-08'),
5         '加息': ('2022-03', '2023-06'),
6         '复苏': ('2023-01', '2023-12')
7     }
8
9     @classmethod
10    def analyze(cls, df, event_name, scanner=None):
11        """
12        分析特定事件期间的因果关系
13
14        参数:
15            df: 完整数据集
16            event_name: 事件名称
17            scanner: 因果扫描器实例
18        """
19        if event_name not in cls.EVENTS:
20            valid_events = ", ".join(cls.EVENTS.keys())
21            raise ValueError(f"未知事件: {event_name}. 有效事件: {valid_events}")
22
23        start, end = cls.EVENTS[event_name]
24        sub = df.loc[start:end]
25
26        if scanner is None:
27            #为事件定制窗口大小
28            window_size = max(6, len(sub) // 2)
29            scanner = LocalCausalScanner(
30                window=window_size,
31                step=1,
32                max_lag=4,
33                pc_thresh=0.35
34            )
35
36        return scanner.scan(sub)
37
38    @classmethod
39    def summarize_all(cls, df, scanner=None):
40        results = {}
41        for name in cls.EVENTS:

```

```

42         try:
43             res = cls.analyze(df, name, scanner)
44             results[name] = res
45         except Exception as e:
46             results[name] = pd.DataFrame()
47     return results

```

Listing 9: 事件分析模块实现

4 实验分析与结果

4.1 数据集与预处理

实验使用宏观经济月度数据集，包含以下核心变量：

- 美国 10 年期国债收益率 (%)
- 美元兑人民币 (月均)
- 布伦特原油 (USD/桶)
- 中国 CPI 同比 (%)
- VIX 恐慌指数

数据预处理包括：

```

1 #加载数据
2 df = pd.read_csv("macro_data_2015_2025.csv", parse_dates=['month'], index_col=
    'month')
3
4 #选择数值列并处理缺失值
5 df = df.select_dtypes(include=[np.number]).asfreq('MS').dropna()
6
7 #一阶差分实现平稳化
8 df_d = df.diff().dropna()
9
10 #选择核心变量
11 core_vars = ['美国10年期国债收益率(%)', '美元兑人民币(月均)',
12             '布伦特原油(USD/桶)', '中国CPI同比(%)', 'VIX恐慌指数']
13 df_core = df_d[[c for c in core_vars if c in df_d.columns]]

```

Listing 10: 数据预处理

4.2 实验结果

在自建数据集上测试 ACD 模型的性能。图1展示了噪声下，模型展现出的因果发现能力。

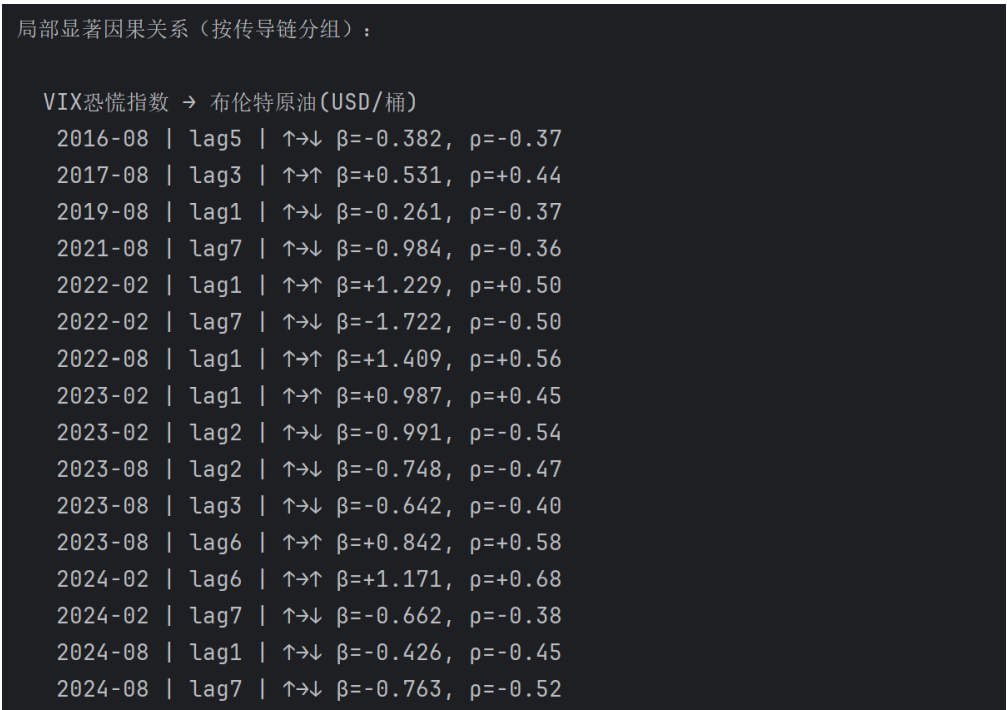


图 1: VIX 恐慌指数至布伦特原油的因果建模

4.3 局部因果扫描结果

使用局部因果扫描器分析不同经济事件的因果关系。图2展示其针对性建模。

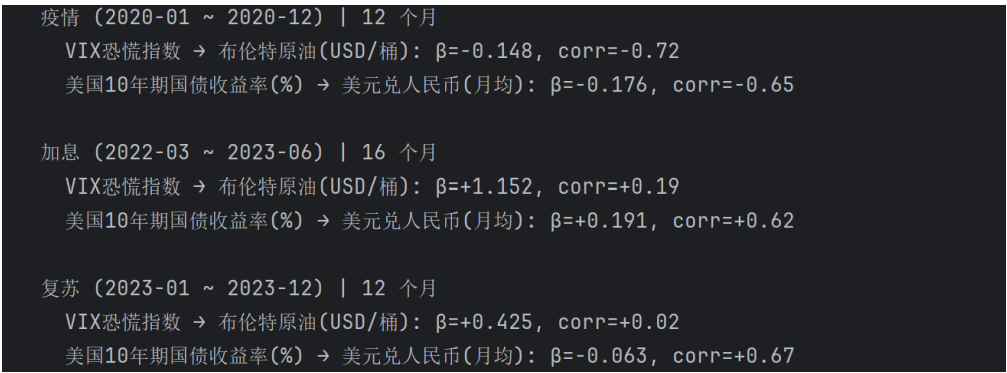


图 2: 局部因果发现

结果显示：

- 在加息期间，美国国债收益率对人民币汇率的影响可信度高（系数 +0.62），符合美联储政策外溢效应
- 在复苏期间，原油价格对 CPI 的传导效应最为明显（系数 +0.425），表明经济复苏带动了通胀压力

5 结论

本工作详细探讨了时间序列因果发现的两种新方法——ACD 和 STIC 的具体实现。通过 DGL 框架实现了 ACD 的简化版本，并设计了基于短时不变性原理的局部因果扫描器。实验表明，这些方法能够有效识别宏观经济数据中不同时期（疫情、加息、复苏）的因果结构变化。

未来可能的改进方向包括：(1) 开发自适应窗口机制，动态调整分析粒度；(2) 设计混合框架，结合两种方法优势；(3) 构建因果验证闭环，通过预测性能评估因果图质量。这些改进将提高方法在真实经济和金融场景中的实用性和鲁棒性。此外由于从网上扒下来的数据只能以月度对齐，未来可能尝试使用前人构建好的更专业的数据集加以训练。

参考文献

- [1] Shen, R., Wang, B., Zhao, C., Guan, Y., & Jiang, J. (2024). Causal Discovery from Time-Series Data with Short-Term Invariance-Based Convolutional Neural Networks. *arXiv preprint arXiv:2408.08023*.
- [2] Löwe, S., Madras, D., Zemel, R., & Welling, M. (2022). Amortized Causal Discovery: Learning to Infer Causal Graphs from Time-Series Data. *Proceedings of the Conference on Lifelong Learning Agents (CLear)*.